

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.

5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I C G Achyuth Venkat (192311102) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure

The form works, but it lacks semantic HTML5 elements that improve structure and accessibility.

Missing elements:

- <header> for the page heading
- <main> for the main content
- <section> to group the registration form
- <label> for each input field
- <footer> (optional) for copyright/contact information

2. Modify the structure to make it standards-compliant

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Online Symposium Registration</title>
```

```
</head>
```

```
<body>
```

```
    <header>
```

```
        <h1>Online Symposium Registration</h1>
```

```
    </header>
```

```
    <main>
```

```
        <section>
```

```
            <form action="/register" method="POST">
```

```
                <label for="name">Name</label>
```

```
                <input type="text" id="name" name="name" required>
```

```
                <label for="email">Email</label>
```

```
                <input type="email" id="email" name="email" required>
```

```
                <button type="submit">Submit</button>
```

```
</form>

</section>

</main>

<footer>

    <p>&copy; 2026 College Symposium</p>

</footer>

</body>

</html>
```

3. Interpret the HTTP request generated during form submission

When the user clicks Submit, the browser sends an HTTP POST request.

Example:

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=John&email=john@example.com

Explanation:

- POST → Sends data securely in the request body.
- /register → Server endpoint.
- Host → Target website.
- Content-Type → Indicates form data format.
- Body → Contains submitted form values.

4. Analyze the HTTP response cycle from server to browser

1. User fills the form.
2. Browser sends a POST request.
3. Server receives and validates data.
4. Server stores the registration details.
5. Server returns an HTTP response.

Example:

HTTP/1.1 200 OK

Content-Type: text/html

Registration Successful

Possible status codes:

- 200 OK – Registration successful
- 201 Created – New registration created
- 400 Bad Request – Invalid input
- 404 Not Found – URL not found
- 500 Internal Server Error – Server problem

5. Suggest improvements for usability and accessibility

- Use descriptive <label> elements.
- Add required validation.
- Use placeholder text if necessary.
- Display validation/error messages.
- Ensure keyboard navigation.
- Use semantic HTML5 elements.
- Make the page responsive using the viewport meta tag.
- Add ARIA attributes where needed.⁴

2. Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags

Issues:

- Missing <!DOCTYPE html>
- <h1> is not closed.
- <p> is not closed.
- Improper nesting of elements.
- No language attribute in <html>.

Incorrect:

<h1>Welcome

<p>About us

Correct:

```
<h1>Welcome</h1>
```

```
<p>About us</p>
```

2. Analyze how different browsers may interpret this structure

Different browsers use **error recovery** techniques.

Possible outcomes:

- Chrome automatically closes missing tags.
- Firefox repairs the HTML differently.
- Safari and Edge may render spacing differently.
- Older browsers may display unexpected layouts.

This causes inconsistent appearance across browsers.

3. Identify missing semantic elements

Missing HTML5 semantic tags:

- <header>
- <main>
- <section>
- <footer>

These improve readability, SEO, and accessibility.

4. Provide a corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Company</title>
```

```
</head>
```

```
<body>
```

```
    <header>
```

```
        <h1>Welcome</h1>
```

```
</header>

<main>

    <section>

        <p>About us</p>

    </section>

</main>

<footer>

    <p>&copy; 2026 Company</p>

</footer>

</body>

</html>
```

5. Suggest techniques to ensure cross-browser compatibility

- Follow W3C HTML standards.
- Validate HTML using the W3C Validator.
- Use CSS Reset or Normalize.css.
- Test in Chrome, Firefox, Edge, and Safari.
- Use feature detection instead of browser detection.
- Avoid deprecated HTML tags.
- Use responsive CSS and Flexbox/Grid.
- Test using browser developer tools.

3. Form Submission Failure

1. Identify issues in the form configuration

Problems:

- Button type is "button" instead of "submit".
- Password is sent using the GET method.
- GET is not suitable for login forms.

Current code:

```
<button type="button">Login</button>
```


Should be:

```
<button type="submit">Login</button>
```

2. Analyze why the HTTP request is not triggered

`type="button"` creates a normal button that performs no action unless JavaScript is added.

Only a **submit** button automatically submits the form.

Therefore, clicking **Login** does not send an HTTP request.

3. Interpret the role of GET method in this scenario

GET sends data through the URL.

Example:

```
/submit?username=John&password=12345
```

Problems:

- Password becomes visible in the URL.
- Stored in browser history.
- Can appear in server logs.
- Can be bookmarked.
- Not secure for authentication.

GET is suitable only for retrieving information, not sending sensitive data.

4. Propose a corrected implementation

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Login</title>
```

```
</head>
```

```
<body>
```

```
<form action="/submit" method="POST">
    <label for="username">Username</label>
    <input type="text"
    id="username"
    name="username"
    required>
    <label for="password">Password</label>
    <input type="password"
    id="password"
    name="password"
    required>
    <button type="submit">Login</button>
</form>
</body>
</html>
```

5. Suggest improvements for secure data transmission

- Use the **POST** method instead of GET.
- Enable **HTTPS** to encrypt data.
- Validate input on both client and server sides.
- Hash passwords on the server (never store plain text passwords).
- Use CSRF protection tokens.
- Implement secure session management.
- Enforce strong password policies.
- Sanitize user inputs to prevent SQL Injection and XSS attacks.